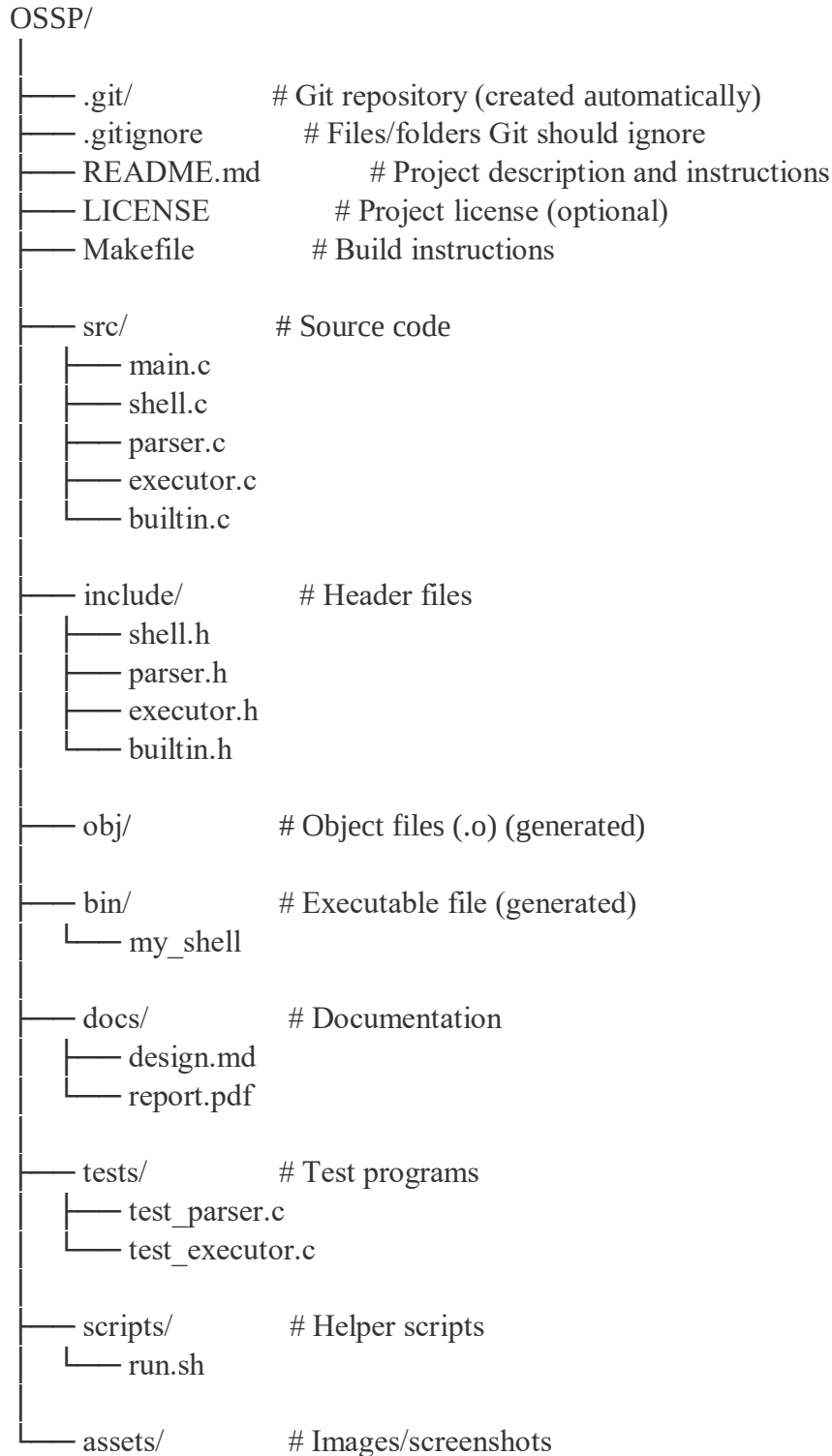


1. To Install Linux VM, Configure GCC, Setup Git Repository, Create Project Structure, Understand Shell Architecture, Build Initial Makefile.

Task -1: Create Project Structure as follows:



└─ architecture.png

Note: For creating this structure use mkdir and touch commands.

```
root@keerthana: ~/OSSP
root@keerthana:~# pwd
/root
root@keerthana:~# cd ~/OSSP
root@keerthana:~/OSSP# pwd
/root/OSSP
root@keerthana:~/OSSP# ls -a
.      .gitignore  README.md  docs      scripts
..     LICENSE  assets     include   src
.git   Makefile   bin        obj        tests
root@keerthana:~/OSSP# tree
.
├── LICENSE
├── Makefile
├── README.md
├── assets
│   └── architecture.png
├── bin
├── docs
│   ├── design.md
│   └── report.pdf
├── include
│   ├── builtin.h
│   ├── executor.h
│   ├── parser.h
│   └── shell.h
├── obj
├── scripts
│   └── run.sh
├── src
│   ├── builtin.c
│   ├── exec_program
│   ├── exec_program.c
│   └── executor.c
```

```
├── scripts
│   └── run.sh
├── src
│   ├── builtin.c
│   ├── exec_program
│   ├── exec_program.c
│   ├── executor.c
│   ├── fork_program
│   ├── fork_program.c
│   ├── main.c
│   ├── parser.c
│   └── shell.c
├── tests
│   ├── test_executor.c
│   └── test_parser.c
```

2. To Analyze process abstraction, execute fork(), understand exec() family, analyze parent-child relationships, inspect process tree, practice system call tracing.

Task-1: Using a manual command understand the fork() and exec() system calls

Task-2: Write a C program to create a new process and accepting the user commands using fork() and exec() system calls.

Example:

```
#include <stdio.h>

#include <unistd.h>

int main() {
    pid_t pid;

    pid = fork(); // Create a new process
    if (pid < 0) {
        printf("Fork failed!\n");
    }
    else if (pid == 0) {
        // Child Process
        printf("This is the Child Process.\n");
        printf("Child PID = %d\n", getpid());
    }
    else {
        // Parent Process
        printf("This is the Parent Process.\n");
        printf("Parent PID = %d\n", getpid());
        printf("Child PID = %d\n", pid);
    }
    return 0;
}
```

```
GNU nano 7.2 fork_program.c
#include <stdio.h>
#include <unistd.h>

int main()
{
    pid_t pid;

    pid = fork();

    if (pid < 0)
    {
        printf("Fork Failed!\n");
    }
    else if (pid == 0)
    {
        printf("This is the Child Process\n");
        printf("Child PID = %d\n", getpid());
    }
    else
    {
        printf("This is the Parent Process\n");
        printf("Parent PID = %d\n", getpid());
        printf("Child PID = %d\n", pid);
    }

    return 0;
}
```

Help **Write Out** **Where Is** **Cut** **Execute** **Location** **Undo** **Set Mark**
Exit **Read File** **Replace** **Paste** **Justify** **Go To Line** **Redo** **Copy**

```
9 directories, 22 files
root@keerthana:~/OSSP# cd src
root@keerthana:~/OSSP/src# nano fork_program.c
root@keerthana:~/OSSP/src# gcc fork_program.c -o fork_program
root@keerthana:~/OSSP/src# ./fork_program
This is the Parent Process
Parent PID = 451
Child PID = 452
This is the Child Process
Child PID = 452
root@keerthana:~/OSSP/src#
```